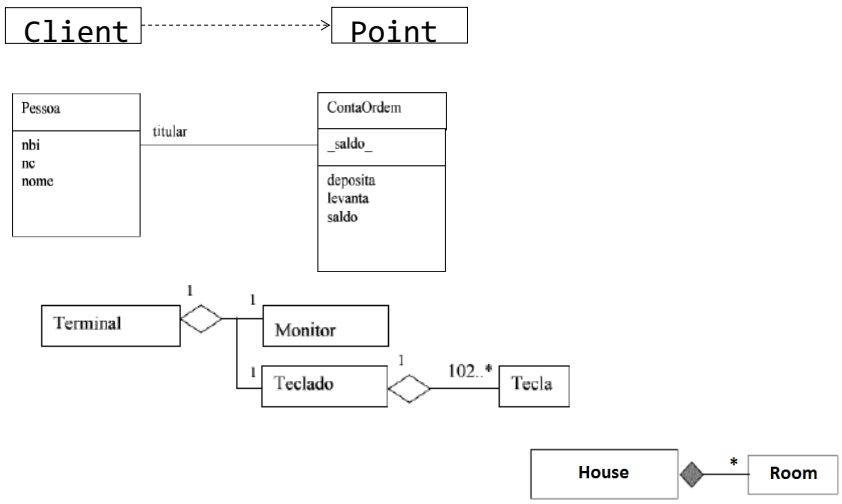
UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

An introduction to UML and object-oriented modeling - II

Readings: ( :chapt. 12)

1

Previously on OOP: UML Relationships between classes



2

Revisiting the notion of class

- From the modeling perspective, a class describes a set of objects with the same semantics (the same role in the program), *i.e.*, with the same:
  - Properties (data structure);
  - Behaviour (functions), and
  - Relationships

Again: How to identify classes?

- Here's one of the simplest technique:

  Identify ***nouns*** in the problem description. For each noun ask:

  1. Is there any data that needs to be saved for this candidate?
  2. Is there any task that this candidate should perform?

  An affirmative answer to any of these questions elect the candidate to a class.

Problem 5: A graphic editor

The task is to develop a graphic editor which is able to maintain and operate 2D geometric figures. More specifically, the editor should be able of, given a square, generate the tangential inner circumference.

The square will be given by 2 suitable points defining its diagonals. For the sake of simplicity, the sides of the square are always parallel to the axis. Under this conditions, 2 points define a square when the distance between coordinates in x are equal to distances coordinates in y.

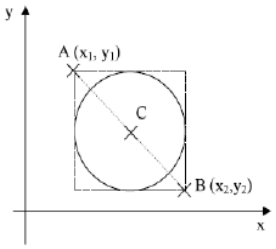
A circumference is represented by its center (a point) and a radius (a float)

Looking for class *candidates*

The task is to develop a *graphic editor* which is able to maintan and operate 2D *geometric figures*. More specifically, the editor should be able of, given a *square*, generate the tangential inner *circumference*.

The *square* will be given by 2 suitable *points* defining its *diagonals*. For the sake of simplicity, the *sides* of the *square* are always parallel to the *axis*. Under these conditions, 2 *points* define a *square* when the distance between *coordinates* in x are equal to the distance *coordinates* in y.

A *circumference* is represented by its *center* (a *point*) and a *radius* (a *float*)



Evaluating classes candidates

- Candidade Point — OK
- Candidate Square — OK
- Candidate Circumference — OK
- ~~Candidate center~~
- ~~Candidate radius~~

7

What class should be responsible for a given task?

- **Q1:** What should be the class responsible for checking whether two points really define a square?
  Re: Square; it is its class invariant.
- **Q2:** What should be the class responsible for computing the distance between two points?
  Re: Point. Why?
- **Q3:** What should be the class responsible for computing the center and radius of the circunference, given a square?

  Re: Two hypothesis: Square or Circunference. In general, we decide in favour of the class that have all the required information to solve the task. In this case, we decide for Square.

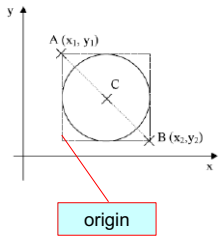
8

How to choose the vars representing the state of a given object?

- In general there are several ways of univocally representing the state of an object.

For example, a square can be represented by:

i) Two representative points defining its main diagonal
ii) Four representative points
iii) Three representative points
iv) The origin point, the side, and an orientation

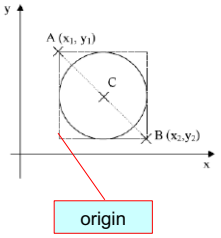


9

How to choose the vars representing the state of a given object?

- In general we choose either
  a) the representation requiring less data (minimal representation) or
  b) the representation more efficient for the operations that the object should reply to

- In this problem, we apply b) above and use the origin point, and the side



10

Check point

- Propose three ways of representing a Point

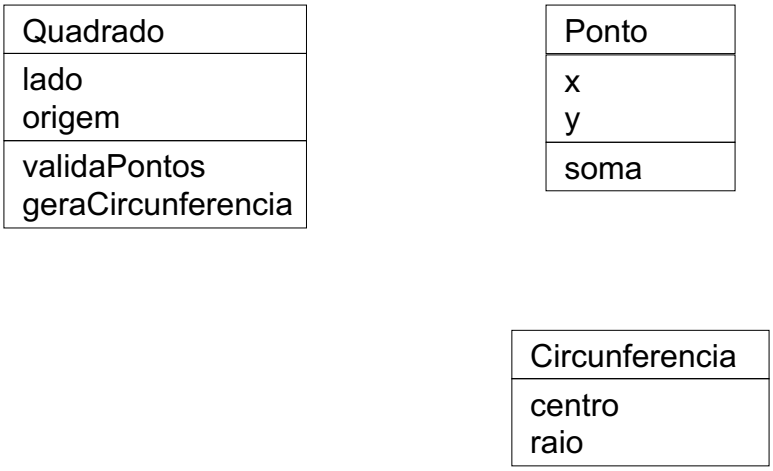
Re:

1. Two variables: x, y (Cartesian representation);
2. _____
3. _____

6-11

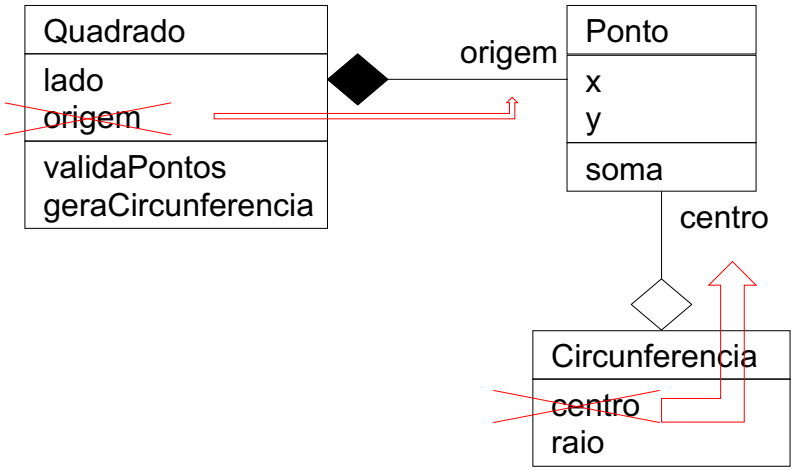
11

An initial class diagram



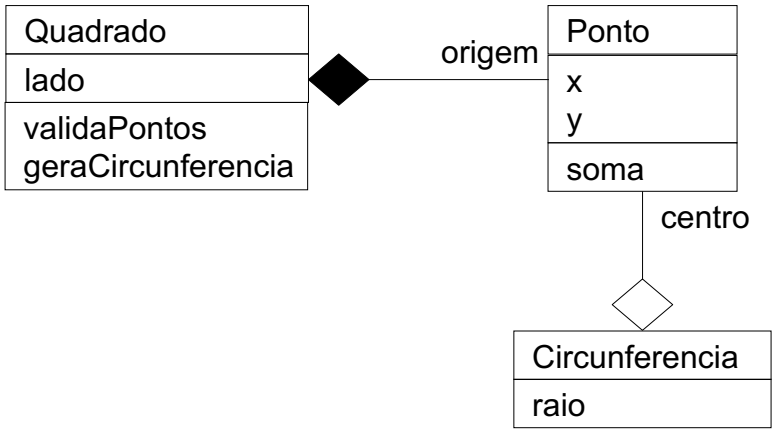
12

An initial class diagram – stressing relationships



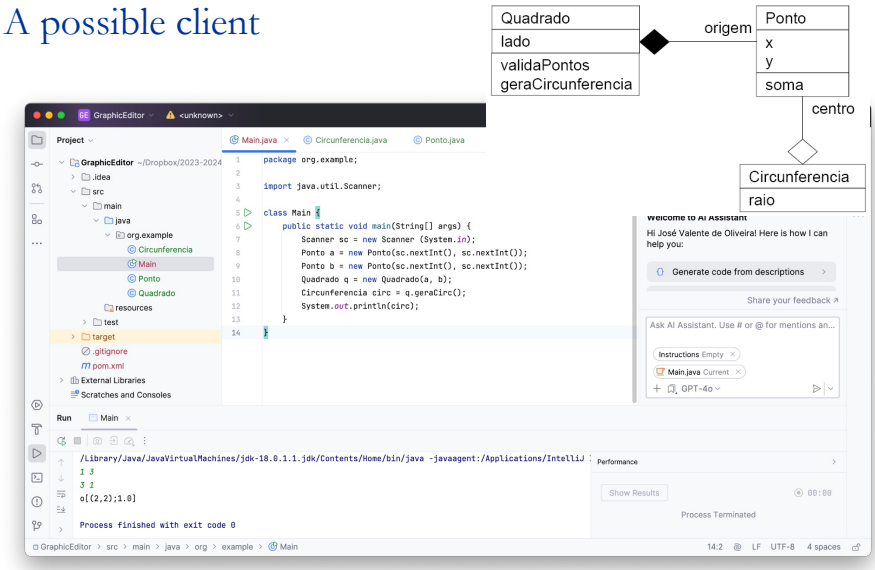
13

From class diagram to code



14

A possible client



15

Refining class Ponto

```
class Ponto {
// …

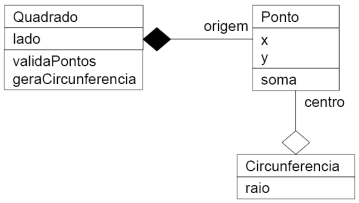
public Ponto soma(Ponto that) {
    int x = this.getX() + that.getX();
    int y = this.getY() + that.getY();
    return new Ponto(x, y);
}

public String toString() {
    return "("+getX() + "," + getY()+")";
    }
}
```

```
Ponto a = new Ponto (1, 2);
Ponto b = new Ponto (2, 0);
Ponto c = a.soma(b);
System.out.println(c); // (3, 2)
```

16

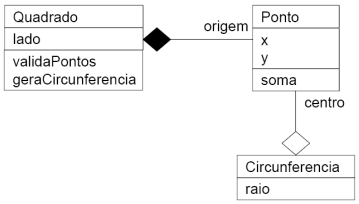
class Circunferencia




```
public class Circunferencia {
    private Ponto centro;
    private float raio;
    public Circunferencia(Ponto c, float r) {
      if (r<=0) {
          System.err.println("ERRO: raio nulo ou negativo.");
          System.exit(0);
      }
      centro=c;
      raio=r;
    }
    public String toString() {
          return centro.toString() + " " + raio;
    }
}
```

17

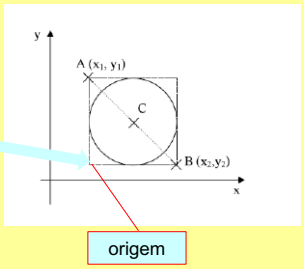
class Quadrado - Constructor




```
public class Quadrado {
private Ponto origem;
private int lado;

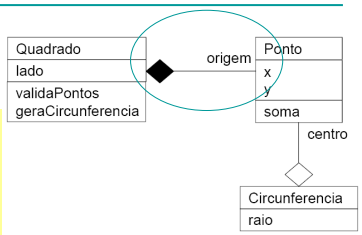
public Quadrado (Ponto A, Ponto B) {
    if (validaPontos(A, B)== false) {
          System.err.println("Erro: os pontos "+ A + " e " + B + " nao definem um quadrado");
          System.exit(0);
    }
    int x = Math.min(A.getX(), B.getX());
    int y = Math.min(A.getY(), B.getY());
    origem = new Ponto(x, y);
    lado = Math.abs(A.getX()-B.getX());
}

public boolean static validaPontos(Ponto A, Ponto B) {
    int lado1 = Math.abs(A.getX()-B.getX());
    int lado2 = Math.abs(A.getY()-B.getY());
    return lado1 == lado2;
}
```



18

class Quadrado - Constructor



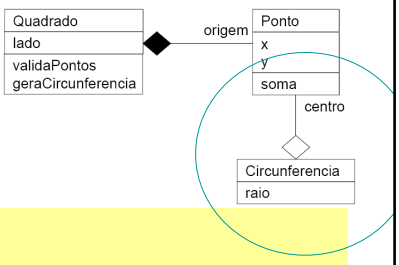

```
public class Quadrado {
private Ponto origem;
private int lado;

public Quadrado (Ponto A, Ponto B) {
    if (validaPontos(A, B)== false) {
            System.err.println("Erro: os pontos "+ A + " e " + B + " nao definem um quadrado");
            System.exit(0);
    }
    int x = Math.min(A.getX(), B.getX());
    int y = Math.min(A.getY(), B.getY());
    origem = new Ponto(x, y);
    lado = Math.abs(A.getX()-B.getX());
}

public boolean static validaPontos(Ponto A, Ponto B) {
    int lado1 = Math.abs(A.getX()-B.getX());
    int lado2 = Math.abs(A.getY()-B.getY());
    return lado1 == lado2;
}
```

19

class Circunferencia




```
public class Circunferencia {
    private Ponto centro;
    private float raio;
    public Circunferencia(Ponto c, float r) {
      if (r<=0) {
            System.err.println("ERRO: raio nulo ou negativo.");
            System.exit(0);
      }
      centro=c;
      raio=r;
    }
    public String toString() {
            return centro.toString() + " " + raio;
    }
}
```

20

Class Quadrado – Conclusion

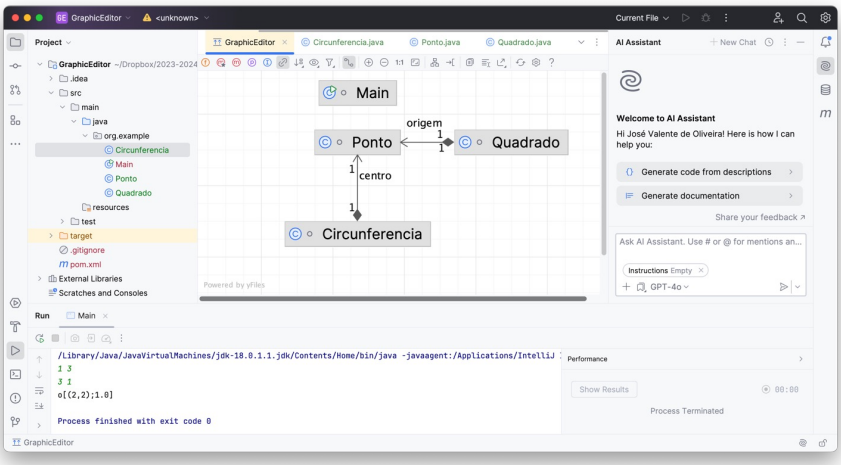
```
public class Quadrado {
// ...

public Circunferencia geraCircunferencia() {
    float raio = lado / 2.0f;
    Ponto centro = origem.soma(new Ponto((int) raio, (int) raio));
    Circunferencia c = new Circunferencia (centro, raio);
    return c;
}

}
```

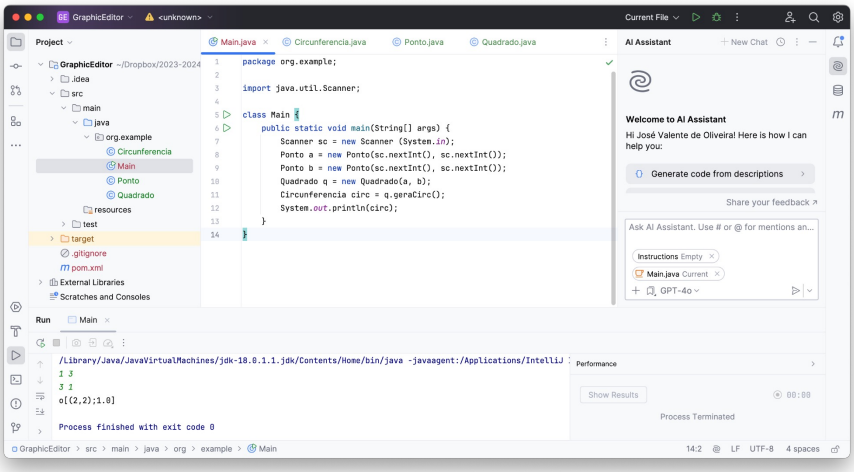
21

Documenting the code with an UML generator



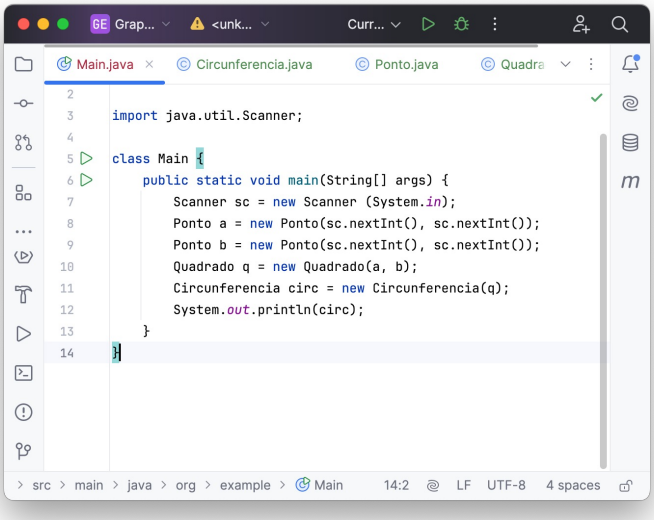
22

The initial client



23

What changes are needed to run this new client code?



24

Changes version 1.0

```
public class Circunferencia {
    private Ponto centro;
    private float raio;
//...
public Circunferencia (Quadrado q) {
    raio = q.getLado() / 2.0f;
    centro = q.getOrigem().soma (new Ponto((int) raio, (int) raio));
    }
}

public class Quadrado {
    private Ponto origem;
    private int lado;
//...
    public int getLado ()          { return this.lado; }
    public Ponto getOrigem ()      { return this.origem; }
}
```

```
Quadrado q = new Quadrado (P1, P2);
//Circunferencia c = q.geraCircunferencia (q);
Circunferencia c = new Circunferencia (q);
```

25

Changes version 2.0

```
public class Circunferencia {
    private Ponto centro;
    private float raio;
//...
public Circunferencia (Quadrado q) {
    Circunferencia c = q. geraCircunferencia ();
    centro = c.centro;
    raio   = c.raio;
    }
}
```

```
Quadrado q = new Quadrado (P1, P2);
//Circunferencia c = q.geraCircunferencia (q);
Circunferencia c = new Circunferencia (q);
```

26

Where are we in OOM?

- Modeling and UML, an Introduction
- Classes and relationship between classes: association, composition, and agregation
- Introduction to UML class and object diagrams
- A complete example